

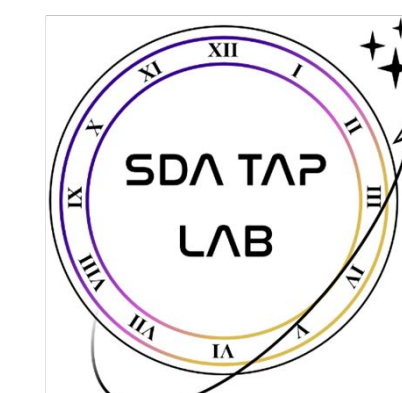


UNIVERSITY OF
CENTRAL FLORIDA

Proposal of OpenEvolve for Uncorrelated Track Processing

Kyle Galang¹, Aurela Broqi¹, Aaron Nogues¹, Ruben Dennis¹, Ezra Stone¹

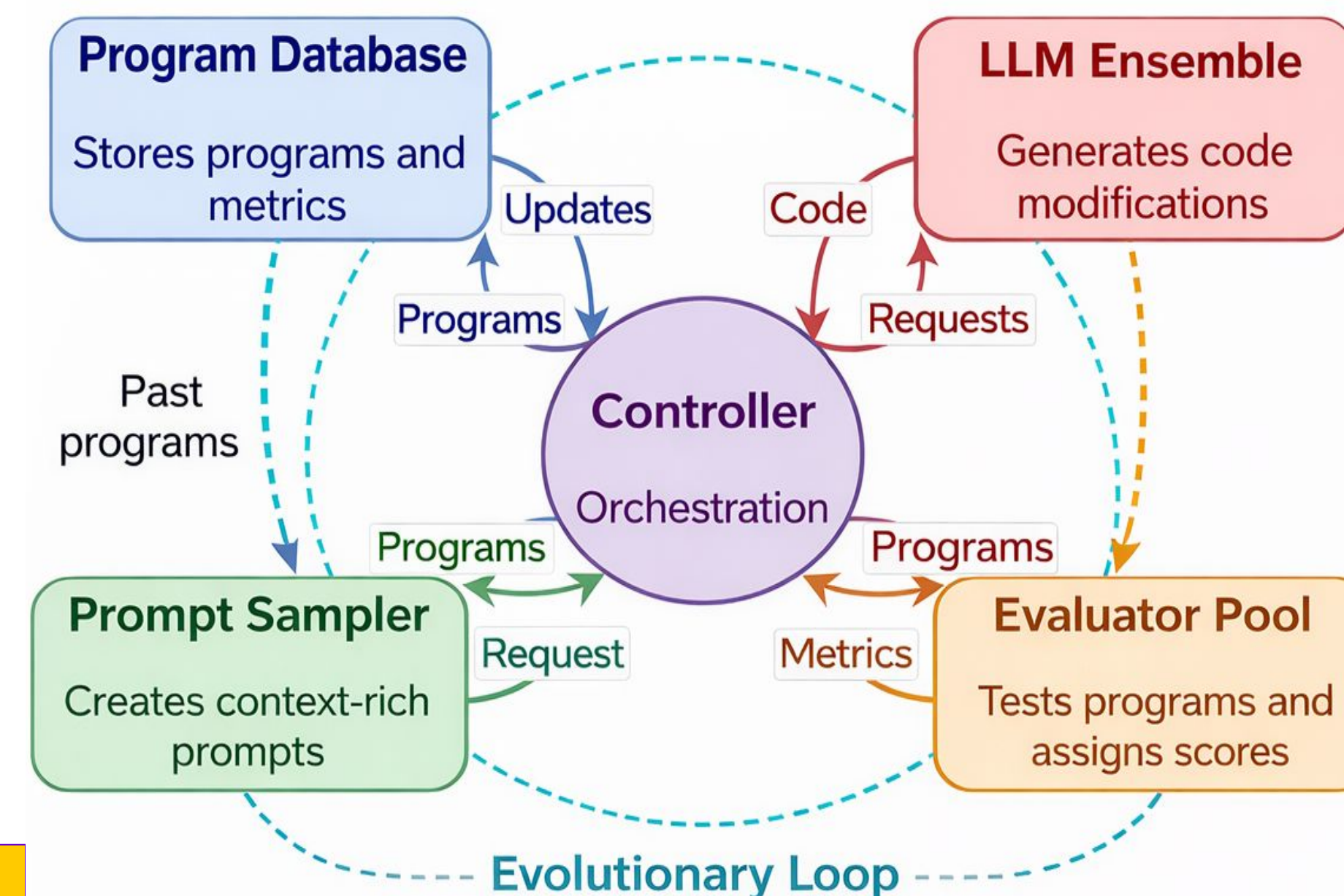
¹University of Central Florida, ²SDA TAP Lab, ³VTARC, ⁴MITRE



Introduction

Uncorrelated Track Processing is especially challenging because few established solutions exist for direct comparison, making benchmarking and processor improvement difficult. This research investigates OpenEvolve, an open-source evolutionary coding agent, as a tool for automating algorithm discovery, testing, and refinement. To support this effort, an analytics console was also developed to provide real-time visibility into performance, enhance transparency and reproducibility, and track program evolution over time.

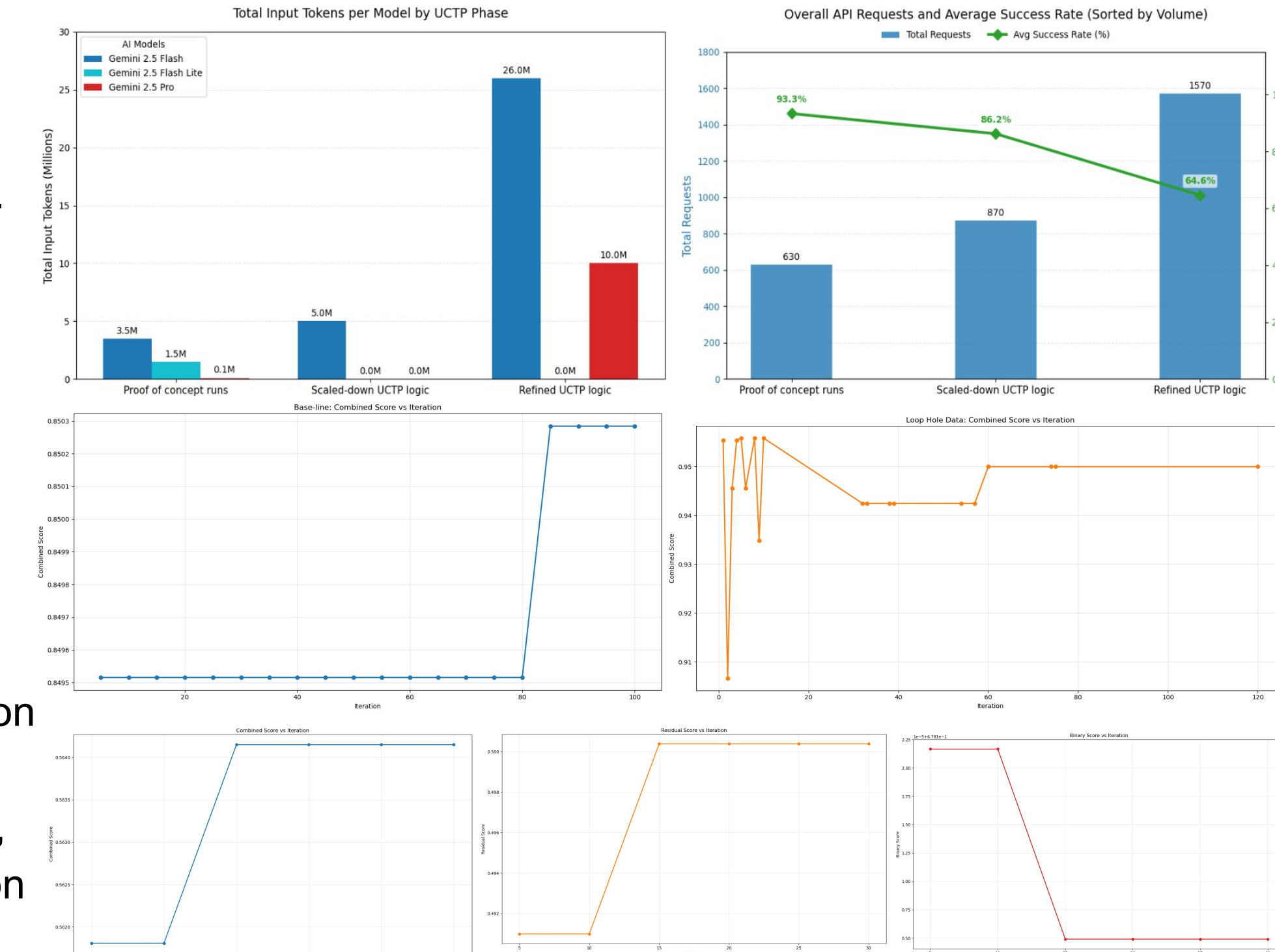
OpenEvolve Architecture



4 Main Components:

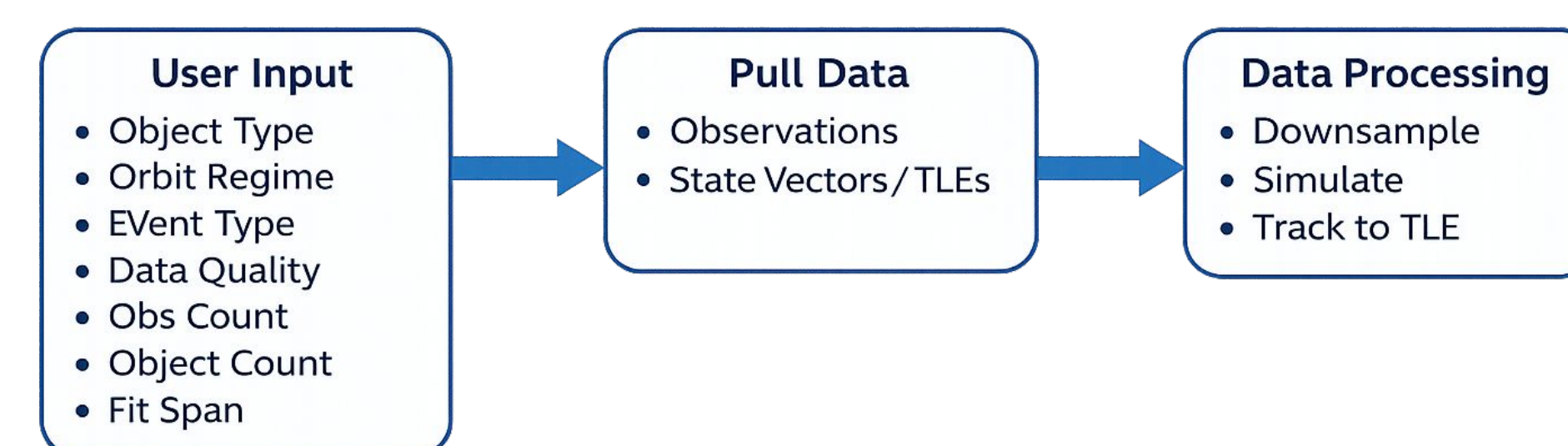
- **Controller Loop:** Orchestrates population management, parallel evaluations, and iteration Control.
- **LLM Ensemble:** Multiple language models (e.g., OpenAI, Gemini, Anthropic) operate as independent “islands.” They periodically exchange elite solutions to prevent premature convergence and encourage diversity.
- **Evaluator:** A user-defined Python script that scores each candidate program’s output. For this project, evaluators will measure regression accuracy or related tracking metrics relevant to UCTP.
- **Database and Artifact Tracker:** Captures all source code versions, execution logs, and performance data to guarantee reproducibility across runs.

Conclusions

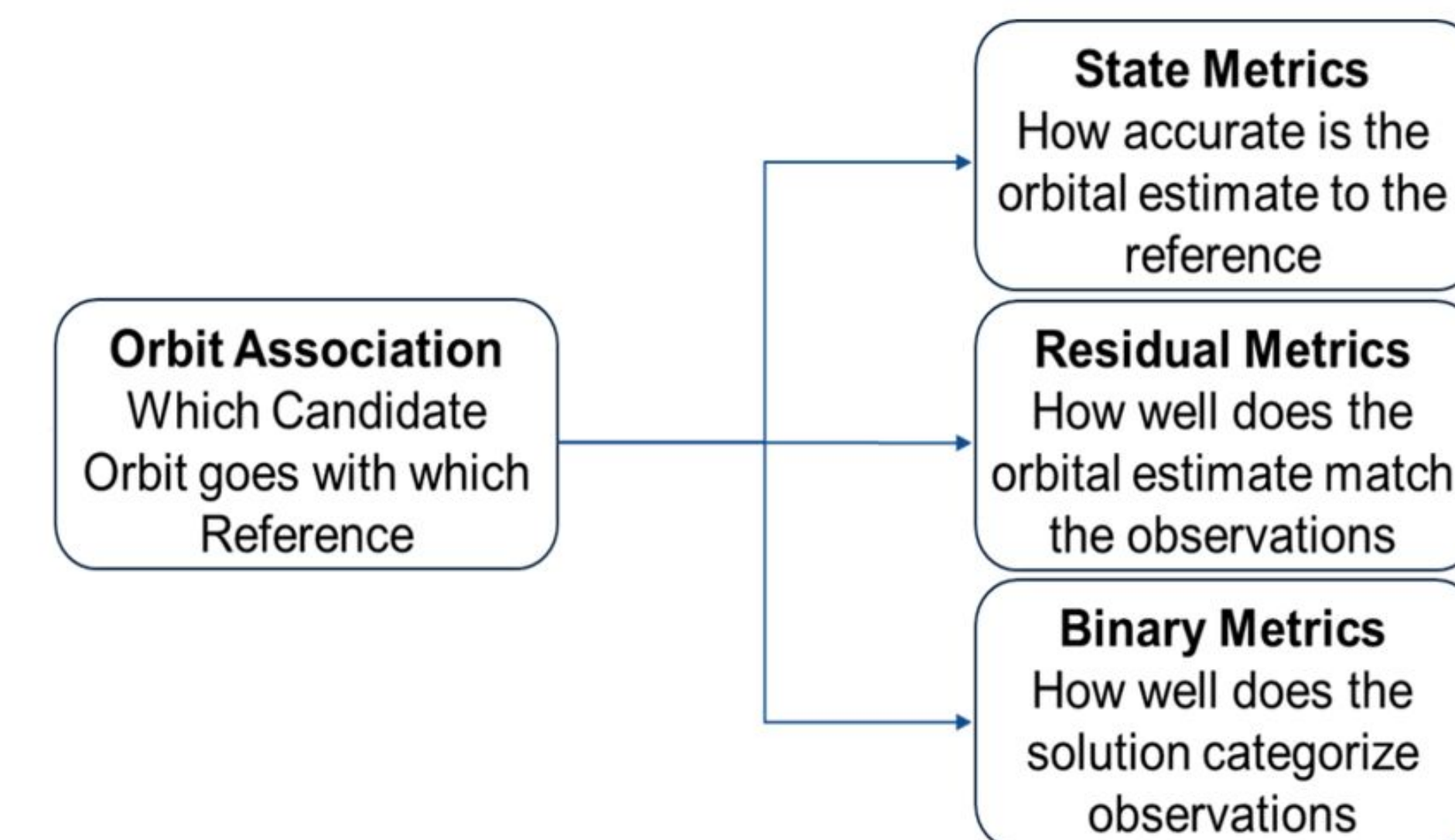


Benchmarking Methodology

Input: UDL data + dataset generation

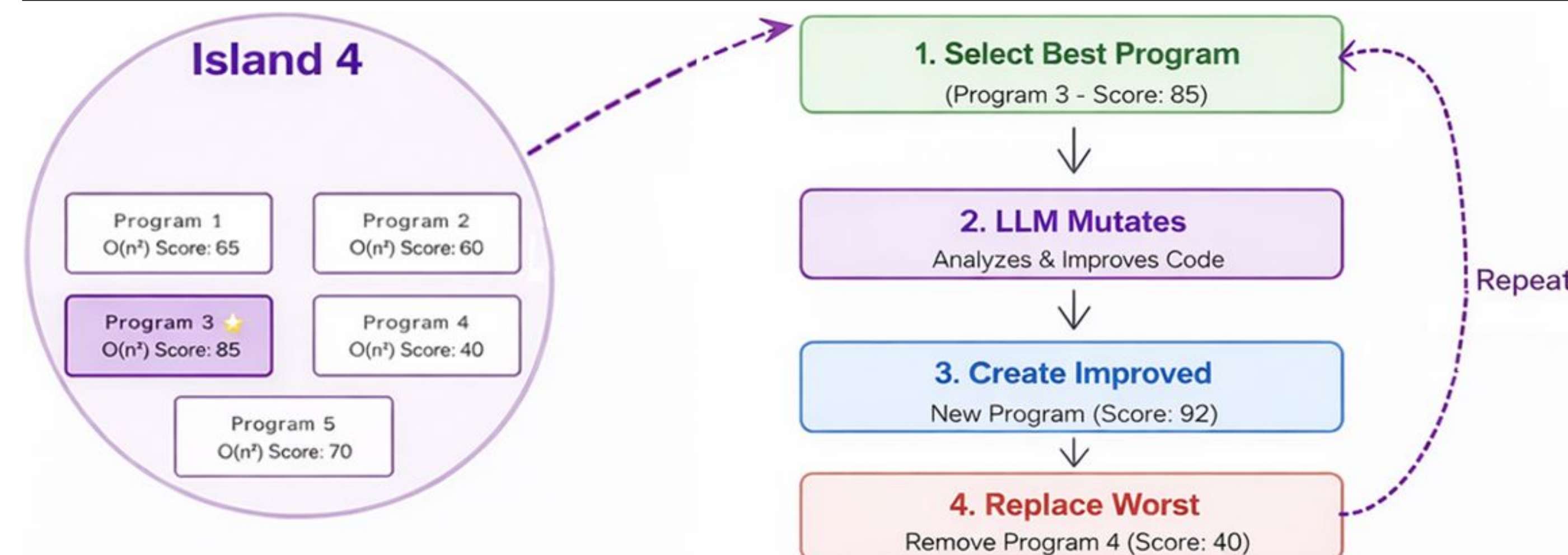


Output: UCT Processor + Evaluation



These are benchmarking schematics for input and output data schemes our team worked with to evolve basic functions of UCTP.

Evolution Process



Instead of having one single large population, multiple islands run parallel to one another. This allows programs to maintain diversity which allows the best solutions to spread through the migration process. Where every n generations, we do migrations where we take the performing programs from one island and move them to another.

Acknowledgements: Louis Caves², Dr. John C. Cline⁴, Melissa hills³, Dr. Richard Leinecker¹

Future Work

- **High Performance Computer Access:** UCTP programs were tested in simplified configurations due to hardware and token limits. HPC resources would enable larger-scale evaluation and localized LLM training.
- **Integrating UCTP Benchmarking into OpenEvolve:** The viability of OpenEvolve for algorithm optimization works. Future plans should be learning how to integrate UCT defined metrics (state, residual, binary) to guide how algorithms are evolving in the future.

References

- [1] Sharma, A. (2025). *OpenEvolve: An open-source evolutionary coding agent* [Software]. GitHub. <https://github.com/algorithmicsuperintelligence/openevolve>
- [2] Shojaei, P., Nguyen, N.-H., Meidani, K., Farimani, A. B., Doan, K. D., & Reddy, C. K. (2025). *LLM-SRBench: A new benchmark for scientific equation discovery with large language models*. arXiv.
- [3] Caves, L., Puchner, M., Stivi, B., Lundin, G., & Smith, C. (n.d.). *Benchmarking and evaluating uncorrelated track processing: Common task framework at the SDA TAP Lab*.
- [4] Oh, S., Russell, S., & Sastry, S. (2005). *Markov chain Monte Carlo data association for multiple-target tracking*.